Module 5

Backtracking & Branch and Bound

General Method

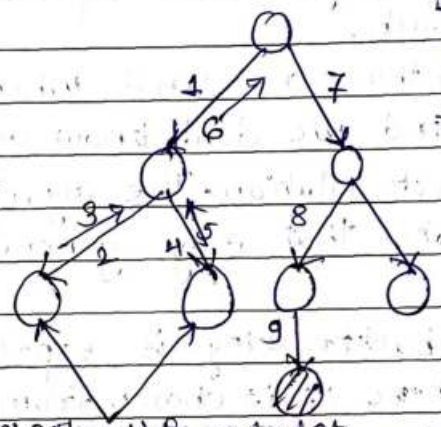
Que Comment on model of computation: Backtracking - 5M

- Backtracking is a recursive approach & it is a refinement of bruteforce method.
- Brute force evaluates all possible candidates that do not satisfy certain constraintes, whereas backtracking limits the search by eliminating the candidates.
- Hence, backtracking algorithms are often faster than brute force approach.
- Backtracking algorithms are used when we have set of choices, and we dont know which choice will lead to a correct solution. The algorithm generates all partial candidates that may generate a complete solution.
- The solution in backtracking is expressed as n-tuple $(x_1, x_2, \cdots x_n)$, where x_i is chosen from the finite set of choices, S_i, Elements in solution tuple are chosen such that it maximize or minimize given criterion function $C(x_1, x_2, \cdots x_n)$
- Let C be the set of constraints on problem P, and let D is the set of all the solutions which satisfy the constraints C, then
  - Finding, if given solution is feasible or not - is the decision problem.
  - Finding best solution - is the optimization problem
  - Listing all feasible solution - is enumeration problem.
- Backtracking systematically searches set of all feasible solutions called solution space to solve the given problems.
- Each choice leads to a new set of partial solutions. Partial solutions called solution space to solve the given problem. are explored in DFS (Depth First Search) order.
- If the partial solution satisfies certain bounding function than the partial solution is expored in Depth First order.

If the partial solution does not satisfy the constraint, it will not explored further. Algo. backtracks from the point & explores to the next possible candidate.

Such processing is convenient to represent using state space tree.

In state space tree, roots represents the initial state before the search begins.



Does not satisfy constraints so dont explore further & backtrack

It keeps exploring the partial solution by adding next possible choice in DFS order and builds the new partial solution. The process continues till partial solution satisfies the given constraints or solution is not found. This is an incremental approach

Fig (1) - Process of Backtracking

A node in state space tree is called promising, if it represents partially constructed solution which may lead to a complete solution.

Non-promising node violates constraints and hence cut down from the further computation

A leaf node is either a non-promising node or it represents a complete solution.

- We consider the backtracking process as finding a particular leaf in the tree. It works as follows:-
- If node N is goal node, then return success & exit.
- If node N is leaf node is a goal not a goal node, then returns failure.
- Otherwise for each childnode C of node N
  - Explores child node C
  - If C is goal node, return "success"
  - Else return failure.

Application of N-Queen problem

1) Task scheduling
2) Computer resource management
3) VLSI testing
4) Deadlock prevention
5) Parallel memory storage schemes
6) Traffic control.

Backtracking (1) N-queens problem (4 queens problem / 8 queens problem)

(2) Graph Coloring problem

(3) Sum of subsets

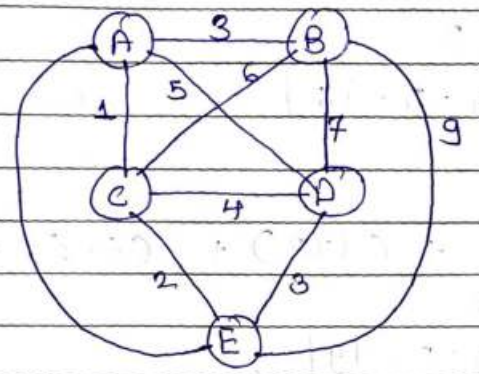
Travelling salesman problem - Branch & Bound.

Problem :- Given n cities, a salesman starts at a specified city (often called source) visit all n-1 cities only once and return to the city where he has started.

Objective :

Find a route through the cities that minimizes the cost thereby maximize the profit.

Que Apply the Branch & Bound Algo. to solve the TSP to the following graph.



Solution, Consier A → source node.

Calculate Lower Bound value

Lower Bound = $s/2$ $s \rightarrow$ sum of two smallest different edges

$$lb = \underset{A}{(1+3)} + \underset{B}{(3+6)} + \underset{C}{\frac{(1+2)}{2}} + \underset{D}{(3+4)} + \underset{E}{(2+3)}$$

$$= \frac{4+9+3+7+5}{2}$$

$$= \frac{28}{2} = 14$$

Initial state space tree

0 | Start, lb=14 |

A-B Compute Lower Bound From vertex A

$$\frac{\overset{A}{(8+1)} + \overset{B}{(3+6)} + \overset{C}{(1+2)} + \overset{D}{(4+3)} + \overset{E}{(2+3)}}{2}$$

$= \frac{28}{2} = 14$ $\boxed{A \to B = 14}$

A-C

$$\frac{\overset{A}{(1+3)} + \overset{B}{(3+6)} + \overset{C}{(1+2)} + \overset{D}{(4+3)} + \overset{E}{(2+3)}}{2}$$

$= \frac{28}{2} = 14$ $\boxed{A - C = 14}$

A-D

$$\frac{\overset{A}{(5+1)} + \overset{B}{(3+6)} + \overset{C}{(1+2)} + \overset{D}{(5+3)} + \overset{E}{(2+3)}}{2}$$

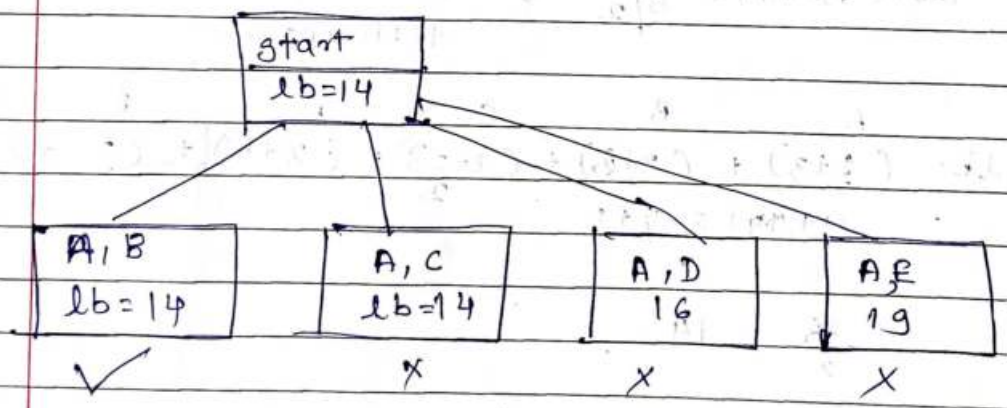
$= \frac{31}{2} = 16$ $\boxed{A - D = 16}$

A-E

$$\frac{(8+1) + (3+6) + (1+2) + (5+3) + (8+2)}{2}$$

$= \frac{38}{2} = 19$ $\boxed{A - E = 19}$

Start space Time



A → B path is choosen

... n-queen problem using backtra-

$B-C \Rightarrow \frac{(3+1)+(3+6)+(6+1)+(3+4)+(2+3)}{2}$

$= \frac{32}{2} = 16$ $\boxed{B-C=16}$

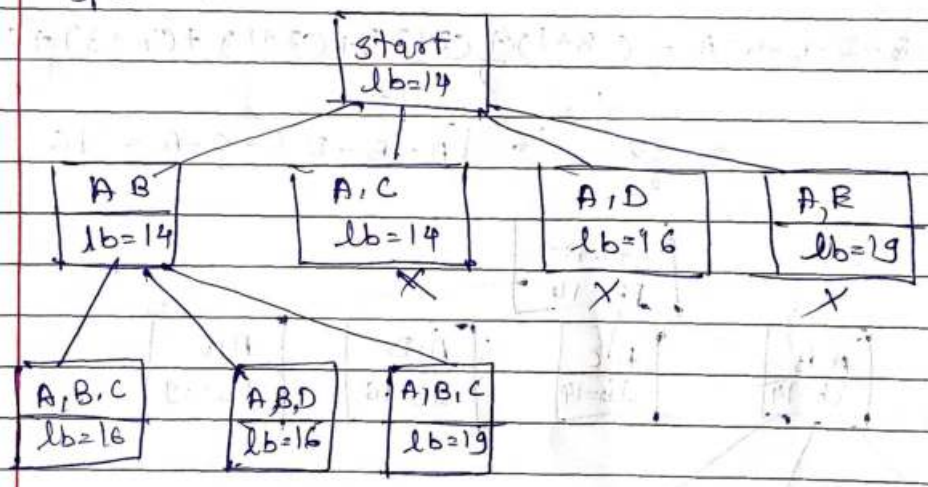
$B-D \Rightarrow \frac{(3+1)+(3+7)+(1+2)+(7+3)+(2+3)}{2}$

$= \frac{32}{2} = 16$ $\boxed{B-D=16}$

$B-E = \frac{(3+1)+(3+9)+(1+2)+(3+4)+(9+2)}{2}$

$= \frac{37}{2} \approx 19$ $\boxed{B-E=19}$

How we will include next level components in space time



A - B - C (Remain vertices D & E)

Compute LB from vertex D & E

$A-B-C-D-E \quad A \Rightarrow \frac{\overset{A}{(3+6)}+\overset{B}{(3+6)}+\overset{C}{(6+4)}+\overset{D}{(4+3)}+\overset{E}{(3+8)}}{2}$

$= \frac{48}{2} = 24$ $\boxed{A-B-C-D-E = 24}$

A - B - C (Remaining vertices E & D)

$$A-B-C-E-D = \frac{\overset{A}{(3+5)} + \overset{B}{(3+6)} + \overset{C}{(6+2)} + \overset{D}{(3+5)} + \overset{E}{(2+3)}}{2}$$

$$= \frac{38}{2} = 19 \quad \boxed{A-B-C-E-D = 19}$$

A - B - D (Remaining C, E)

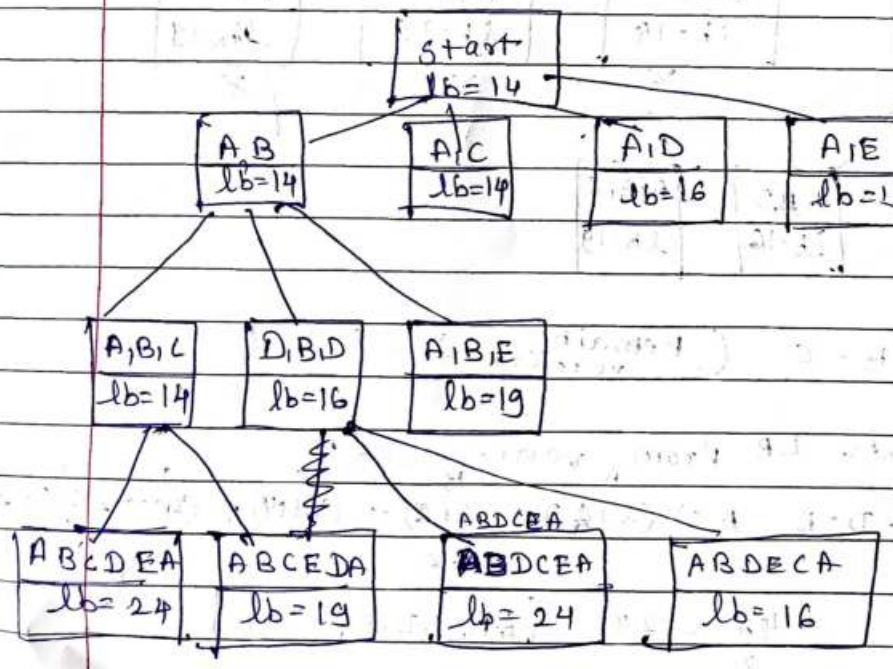
$$A-B-D-C-E-A = \frac{\overset{A}{(3+8)} + \overset{B}{(3+7)} + \overset{C}{(4+2)} + \overset{D}{(4+\ )} + \overset{E}{(2+8)}}{2}$$

$$= \frac{48}{2} = 24 \quad \boxed{A-B+D-C-E-A = 24}$$

A - B - D (Remaining E, C)

$$A-B-D-E-C-A = \frac{(3+1)+(3+7)+(2+1)+(7+3)+(3+2)}{2}$$

$$= \frac{32}{2} = 16 \quad \boxed{A-B-D-E-C-A = 16}$$



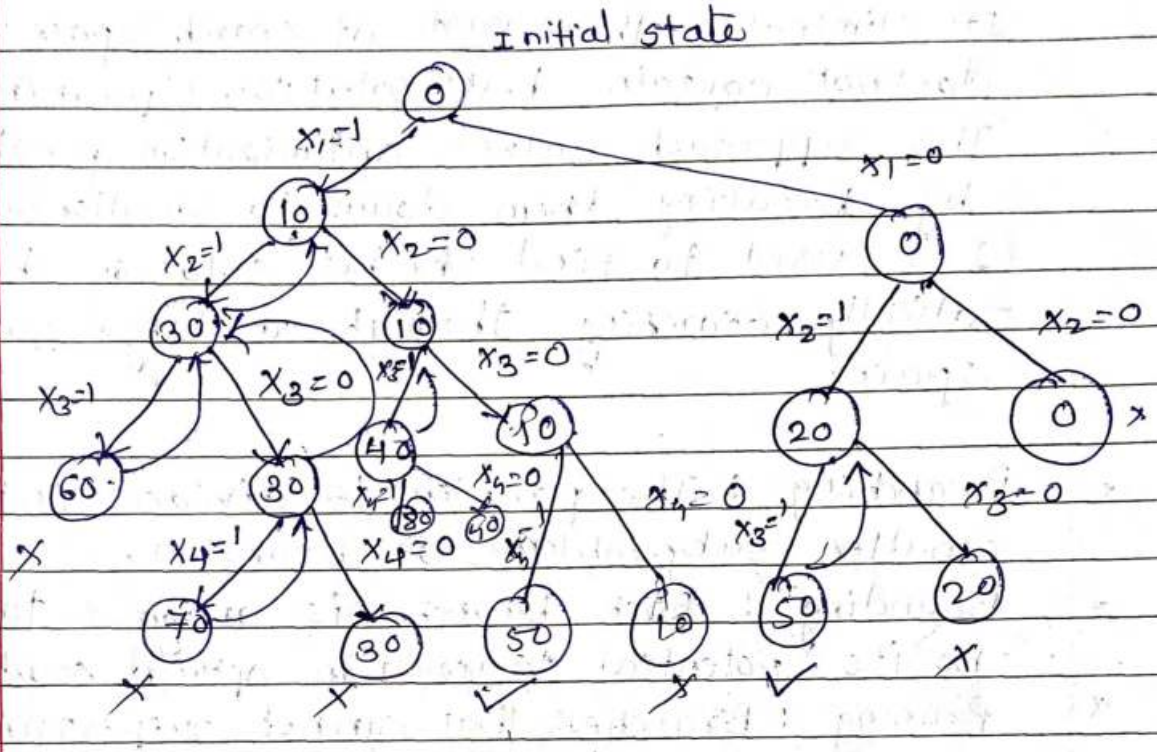
TSP path = A - B - D - E - C - A

cost = 16

Sum of Subset :-

$A = \{ \overset{x_1}{10}, \overset{x_2}{20}, \overset{x_3}{30}, \overset{x_4}{40} \} \rightarrow \{10, 40\}, \{20, 30\}$

Generate subsets so that sum = 50



In the sum of subsets problem, there is a given set with some non-negative integer elements. And another sum value is also provided, our task is to find all possible subsets of the given set whose sum is the same as the given sum value.

Branch & Bound Algorithm

- Branching is the process of generating subproblems.
- Bounding refers to ignoring partial solutions that cannot be better than current best solution.
- It is a search procedure to find optimal solution.
- It eliminates those parts of search space which does not contain best solution (pruning)
- This approach solves optimization problems by breaking them down to smaller subproblems. It's used to find the best solution by system-atically searching through a large solution space.

* Branching : The problem is divided into smaller subproblems, or branches.
* Bounding : Each branch is analyzed to determi-ne its potential to yield an optimal solution
* Pruning : Branches that cannot outperform the current Best solutions are discarded.

Applications:

1. Logistics optimization
2. Resource allocation
3. Manufacturing optimization
4. Telecommunications network design
6. Network design
7. Portfolio optimization
8. Airline scheduling

Benefits:

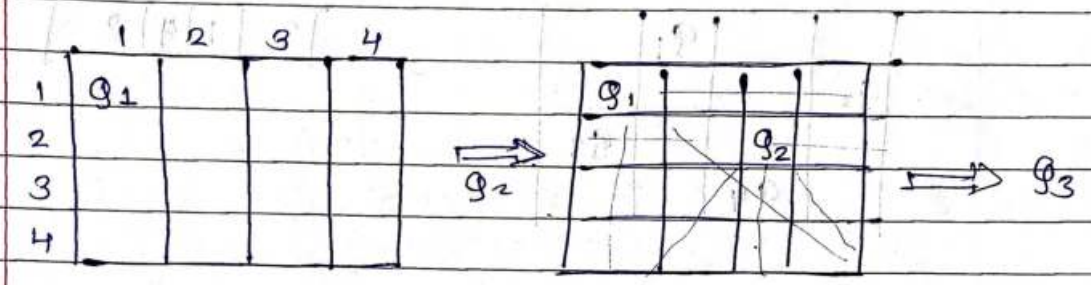
Effective at reducing the search space, especially for NP-complete problems. However, it can be time consuming, and parallelization can be difficult.

Que Give the algorithm to solve N-queen problem using backtracking. Give any two solutions for N-queen problem.

N Queen problem

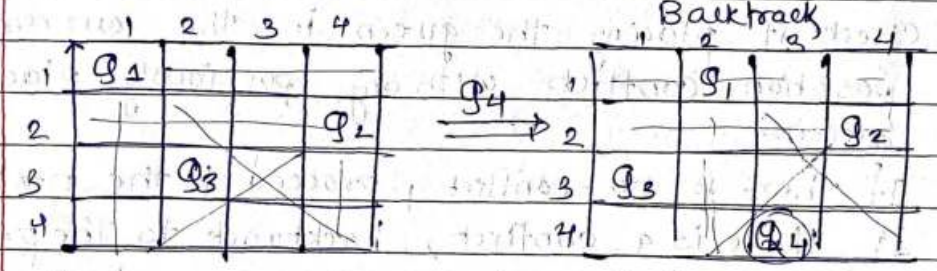
Given an integer n, the task is the solution to the N-queen problems, where n queens are placed on an n*n chessboard such that no two queens can attack each other.

4 queen problem



Q_1 Q_2

we need to Backtrack



Q_3

Backtrack Q_1

Desired solution

2	4	1	3

The 4-queens problem is a classical combinational problem where you place 4 queen on a 4X4 chessboard so that no two queens are on the same row, column or digonal.

You can use backtracking algorithm to find solutions to the 4-queens problem.

In backtracking, you place a queen at different position on the chessboard & find the right arrangement

You can use a board array to determine if two queens share same row.
You can use pointer to the current column & a for loop to check for row conflicts.
You can use propogation to exclude some squares.

The other solution

		q_1	
q_2			
			q_3
	q_4		

3	1	4	2

Algorithm :-

1. Place a queen in the 1st column of the first row.
2. Check if placing the queen in the current position conflicts with any previously placed queens.
3. If there is no-conflict, proceed to the next column. If there is a conflict, backtrack to the previous column and try placing the queen in the next row.
4. Repeat steps 2 & 3 for all columns.
5. If all queens are placed successfully without conflicts, a solution is found.
6. If no solution is found after exploring all possibilities, backtrack to the previous column & try placing the queen in the next row.

Time Complexity :
Worst case. $O(N!)$
because there are $N!$ possible permutation of queen on the board.

Explain 8 queen problem using backtracking. (10M)

1. Initialize:

Create an empty 8x8 chessboard

Start with the first row (row = 0)

2. Recursive Backtracking (starting from row 0 → 7)

- For each row, try placing a queen in every column
- For each column, in the current row, check if placing the queen in that column is safe.
  - No queen already exists in the same column (i.e. check previous rows)
  - No queen exists in the same diagonal (both directions)
- If placing the queen at (row, col) is safe:
  - Place the queen at (row, col).
  - Move to the next row (row+1) and attempt to place queens recursively.
- If placing a queen leads to an invalid state (i.e. no valid configuration is possible in the subsequent rows), backtrack
  - Remove the queen from (row, col)
  - Try the next column in the current row.

3. Base Case:

- If you reach the last row (row 7), and successfully place all 8 queens, print the solution

Graph Coloring

Que Explain Graph coloring problem using backtracking. while algorithm for the same. – 10M

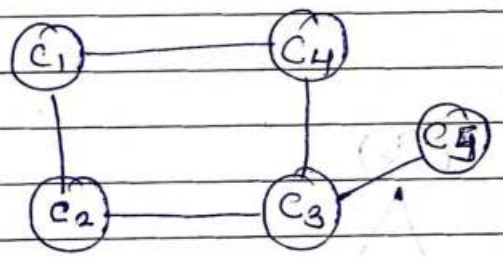
→ Graph coloring is the problem of coloring vertices of a graph in such a way so that no two adjacent vertices have the same colour. Also called as vertex colouring problem.

If coloring is done using at most k colors, it is called k-coloring.

The smallest no. of colours required for coloring graph is called its chromatics number & finding it is a NP-complete problem.

Applications

(i) Design a timetable
(ii) Sudoku
iii) Register allocation in compiler
iv) Map colouring
v) Mobile radio frequency assignment

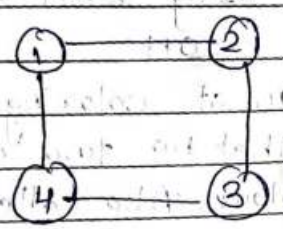


	C_1	C_2	C_3	C_4	C_5
C_1	0	1	0	1	0
C_2	1	0	1	0	0
C_3	0	1	0	1	1
C_4	1	0	1	0	0
C_5	1	0	1	0	0

The input to the graph is adjacency matrix representation of the graph. Value $M(i,j) = 1$ in the matrix represents there exists an edge between vertex i and j. A graph and its adjacency matrix representation as shown in fig.

The problem can be solved by assinging a unique colour to each vertex, but this situation is not optimal. It may be possible to color the graph with colours less than $|V|$.

Graph → No of colors = 3 (c_1, c_2, c_3)

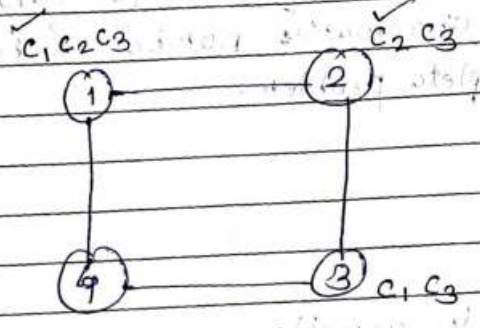


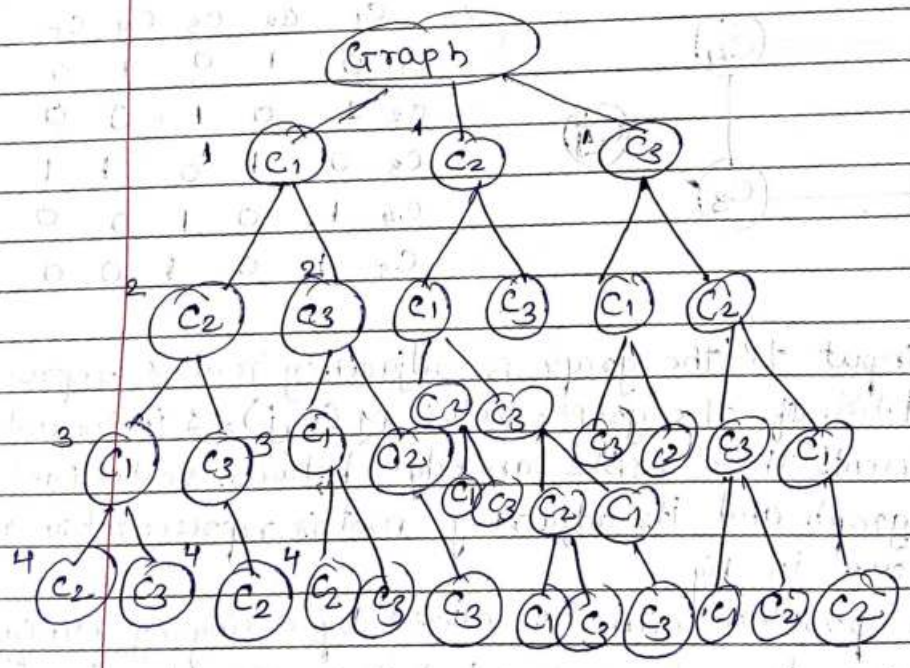
constraints :

1) Explicit constraints :
fixed no of color are used

2) Implicit constraints :
connected vertices should not be same color.

Solution space Tree





No. of possible solution = 6 × 3 = 18
leaf node Trees solutions

15 – Puzzle Problem

Cost function:

Each node X in the search tree is associated with the tree cost function & is useful for determining the next node (E node: is a live node whose children are currently being explored. In other words, an E-node is a node currently being expanded)

The next E-node is one with least cost.

The cost function is defined as:

$C(X) = g(X) + h(X)$

$g(X)$ = cost of reaching the current node from the root.

$h(X)$ = cost of reaching an answer node from X.

Ideal cost function for 15-puzzle algorithm:

- We assume that moving one tile in any direction will have 1 unit cost. Keeping that in the mind, we define cost function for 15 puzzle algo. as below:

$C(X) = f(X) + h(X)$

$f(X) \rightarrow$ length of the path from root to x (the no. moves so far)

$h(X) \rightarrow$ no. of non-blank tiles not in their goal goal position (no. of misplaced tiles)

There are at least $h(X)$ moves to transform state X to a goal state.

Initial state (An arrangement)

1	2	3	4
5	6		8
9	10	7	11
13	14	15	2

Goal state (Goal arrangement)

1	2	3	4
5	6	7	8
9	10	11	12
	13	14	15

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

1	2	3	4
5	6		8
9	10	7	11
13	14	15	12

1+1+1+1 = 4 Up

Right

1+1+1+1 = 4

Down

1+1 = 2

Left

1+1+1+1 = 5

1	2		4
5	5	3	8
9	10	7	11
13	14	15	12

Bound

1	2	3	4
5	6	8	
9	10	7	11
13	14	15	12

Bound

1	2	3	4
5	6	7	8
9	10		11
13	14	15	12

Branch

1	2	3	4
5		6	8
9	10	7	11
13	14	15	12

Bound

1

Right

1	2	3	4
5	6	7	8
9	10	11	
13	14	15	12

1+1+1 = 3

Down

1	2	3	4
5	6	7	8
9	10	15	11
13	14		12

1+1+1 = 3

Left

1	2	3	4
5	6	7	8
9		10	11
13	14	15	12

UP

1+1

1	2	3	4
5	6	7	
9	10	11	8
13	14	15	12

Down

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

Goal

eg :-

12	2	1	15
7	9	10	4
11	5	6	8
	14	13	3

⟹

15	14	13	12
11	10	9	8
7	6	5	4
3	2	1	